



1.18 Exploring HttpServletRequest and HttpServletResponse

HttpServletRequest

- ◆ When you send the request using http protocol that request is called as HttpRequest.
- ◆ HttpRequest contains two parts:
 - HttpRequest Headers.
 - HttpRequest Body.
- ◆ When you send the request with Http method GET then parameter data will be converted to Query String and that Query String will be attached to the URL as follows:

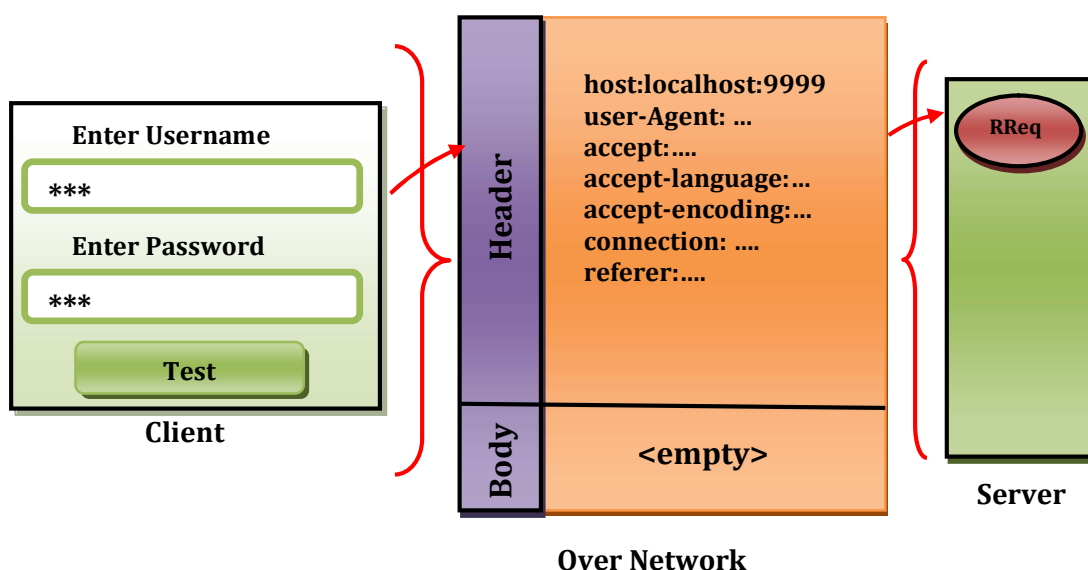
`http://localhost:9999/myjlcapp/login.html`

`http://localhost:9999/myjlcapp/login.jlc?username=sri&password=jlcindia`

<code>http://localhost:9999/Lab14/login.jlc</code>	URL
<code>/Lab14</code>	CONTEXT PATH
<code>http</code>	PROTOCOL
<code>localhost</code>	HOST
<code>9999</code>	PORT
<code>/Lab14/login.jlc</code>	URI
<code>username=sri&password=jlcindia</code>	Query String

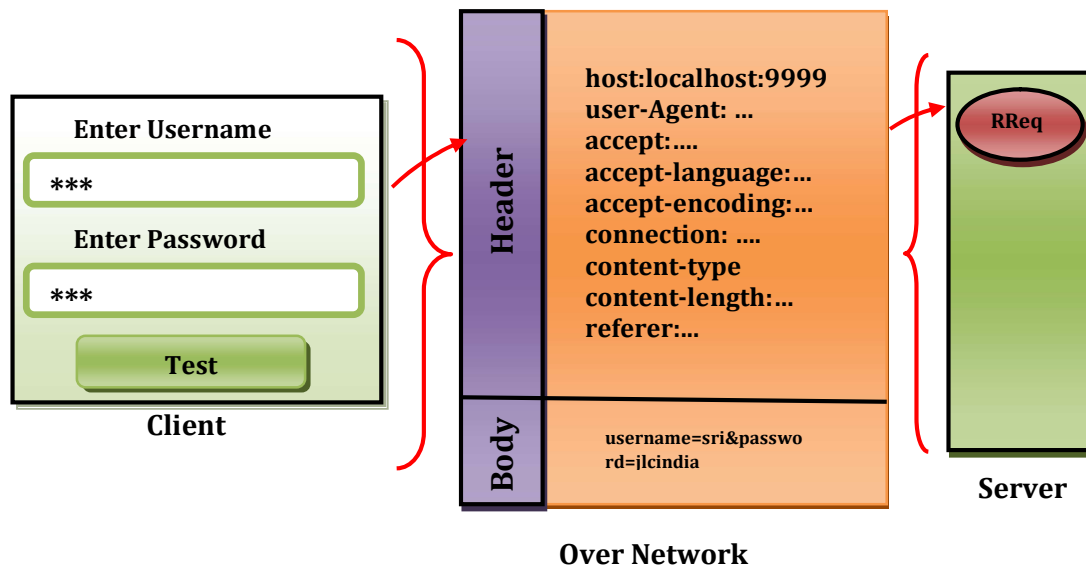
- ◆ In the case Http method GET, this data will be placed in HttpRequest Headers and HttpRequest Body is empty.

`http://localhost:9999/myjlcapp/login.jlc?username=sri&password=jlcindia`



- When you send the request with Http method POST then parameter data will be converted to Query String and that Query String will be placed in HttpRequest Body. Then URL is like

http://localhost:9999/myjlcapp/login.jlc



GET	POST
When you send the request with Http method GET, this data will be attached to URL as Query String.	when you send the request with Http method POST, this data will be placed in Http Request Body.
Using GET, you can send only limited amount of Data.	Using POST, you can send unlimited amount of Data.
GET is not secure because data will be visible in the URL.	POST is more secure.

- At Serverside, Container is responsible for creating HttpServletRequest and HttpServletResponse objects and Initializes those objects with the data.
- HttpServletRequest contains the following information
 - Request Parameters
 - Request Headers
 - Request Cookies
 - Other Information.



- ♦ Following are Http Methods supported
 - GET
 - POST
 - PUT
 - DELETE
 - HEAD
 - PATCH
 - TRACE
 - OPTIONS

HttpServletResponse

- ♦ When you receive the response using http protocol that response is called as HttpServletResponse.
- ♦ HttpServletResponse contains two parts:
 - HttpServletResponse Headers.
 - HttpServletResponse Body.

HttpServletResponse Headers.

Following are some the Response Headers from Server.

200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Content-Length:55KB
Last-Modified: Mon, 18 Jul 2016 02:36:04 GMT
Server: Apache
Set-Cookie: mykey=myvalue; Max-Age=31449600;
etc

HttpServletResponse Body.

- ♦ Any type of output sent by the Server side component will be written to the HttpServletResponse.
- ♦ All MIME types Allowed in HttpServletResponse Body.
- ♦ Some of the MIME Types includes
 - text/html
 - text/plain
 - application/xml
 - application/json
 - application/pdf
 - etc
- ♦ HttpServletResponse contains the following information
 - Response Headers
 - Response Stream in terms of Byte and Character.



demo1-mvc-boot: Spring MVC with Boot (using JSP)

Working Steps:

A) Setup the Project

1) Create the maven Project.

- a) Open the Eclipse required Workspace
- b) Select New -> Maven Project.
- c) Check the checkbox for create Simple Project(skip the Archetype selection) and Click on Next Button.
- d) Provide the following
Group Id: com.coursecube.springboot
Artifact Id : **demo1-rest-boot**
Version : 1.0
packaging : jar
Name : **demo1-rest-boot**
and Click On finish button.

2) Remove the following two Source Folders.

- a. src/test/java
- b. src/test/resources

3) Create application.properties under src/main/resources

4) Update **application.properties** as follows

spring.application.name=my-hello-application
server.port=12345

5) Update **pom.xml** by adding the following dependencies.

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-devtools</artifactId>  
</dependency>
```



6) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

Select the Project , right click and then select Maven -> Update Project.

7) Create the Package called **com.jlcindia.springboot** under src/main/java

8) Write Boot Main Class – MyBootApplication.java

@SpringBootApplication

```
public class MyBootApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(StartMyBootApplication.class, args);  
    }  
}
```

9) Run the Boot Application

You will see – Whitelabel Error Page means DS is Ready.

B)Develop RESTful Web Service

10) Write Resource Representation Class called **Hello**

a. **Hello.java** (com.jlcindia.springboot)

11) Create a Resource Controller called HelloController

a. **HelloController.java** (com.jlcindia.springboot)

C) Test Rest API with Browser.

12) Hit the following urls from Browser

http://localhost:12345/myhello

http://localhost:12345/myhello/Srinivas

http://localhost:12345/myhello/name/Srinivas

http://localhost:12345/myhello/name/Srinivas/message/Hello Guys!!!

http://localhost:12345/myhello1/name/Srinivas

http://localhost:12345/myhello1



D) Test Rest API with CURL.

13) Download CURL from <https://curl.haxx.se/download.html>

You will get zip: curl-7.67.0.zip

14) Extract to E: drive.

Now curl home is E:\curl-7.67.0

15) Goto E:\curl-7.67.0 and type the following

curl -v GET localhost:12345/myhello

curl -v GET localhost:12345/myhello/Srinivas

curl -v GET localhost:12345/myhello/name/Srinivas

curl -v GET localhost:12345/myhello/name/Srinivas/message/Hello Guys!!!

curl -v GET localhost:12345/myhello1/name/Srinivas

curl -v GET localhost:12345/myhello1

E) Test Rest API with Postman

try yourself

Spring Boot-REST Example

demo1-rest--boot: Files required

1. MyBootApplication.java	2. Hello.java
3. HelloController.java	4. application.properties
5. pom.xml	

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```



2. Hello.java

```
package com.jlcindia.springboot;

import javax.xml.bind.annotation.XmlRootElement;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@XmlRootElement
public class Hello {

    private int id;
    private String name;
    private String message;

    public Hello() {}
    public Hello(int id, String name, String message) {
        super();
        this.id = id;
        this.name = name;
        this.message = message;
    }

    //Setters and Getters

    @Override
    public String toString() {
        return "[" + id + ", " + name + ", " + message + "]";
    }

}
```

3. HelloController.java

```
package com.jlcindia.springboot;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
@Controller
public class HelloController {

    @RequestMapping(value="/myhello", method = RequestMethod.GET)
    @ResponseBody
    public String getMessage1() {
        System.out.println("HelloController - getMessage1()");

        String msg = "Hello , I am getMessage1";

        return msg;
    }

    @RequestMapping(value="/myhello/{name}", method = RequestMethod.GET)
    @ResponseBody
    public String getMessage2(@PathVariable String name) {
        System.out.println("HelloController - getMessage2()");

        String msg = "Hello "+name+" !!! - I am getMessage2";

        return msg;
    }

    @RequestMapping(value="/myhello/name/{name}",method =
    RequestMethod.GET,produces = {"application/json"})
    @ResponseBody
    public Hello getMessage3(@PathVariable String name) {
        System.out.println("HelloController - getMessage3()");

        Hello hello =new Hello(101,name, "I am getMessage3" );

        return hello;
    }

    @RequestMapping(value="/myhello/name/{name}/message/{msg}",
    method = RequestMethod.GET,produces = {"application/xml"})
    @ResponseBody
    public Hello getMessage4(@PathVariable String name,@PathVariable String msg) {
        System.out.println("HelloController - getMessage4()");

        Hello hello =new Hello(109,name, msg);

        return hello;
    }
}
```



```
@RequestMapping(value="/myhello1/name/{name}",
method = RequestMethod.GET, produces = {"application/xml","application/json"})
@ResponseBody
public Hello getMessage5(@PathVariable String name) {
    System.out.println("HelloController - getMessage5()");

    Hello hello =new Hello(12345,name, "I am getMessage5");

    return hello;
}

@RequestMapping(value="/myhello1", method = RequestMethod.GET)
@ResponseBody
public Hello getMessage6(@RequestParam(required = false) String name) {
    System.out.println("HelloController - getMessage6()");

    Hello hello =new Hello(12345,name, "I am getMessage6");

    return hello;
}
}
```

4. application.properties

```
spring.application.name=my-hello-application
server.port=12345
```

5. pom.xml

```
<project ....>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo1-rest-boot</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>demo1-rest-boot</name>
<description>Spring REST with Boot</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName>myjlcapp</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```